CSS Column-Count

Complete Guide: Multi-Column Text Layout

Contents

1 Introduction

The CSS `column-count` property creates multi-column layouts for text content. It automatically divides content into a specified number of columns, enabling magazine-style and newspaper-like text layout on web pages.

1.1 Multi-Column Benefits

Multi-column layouts provide:

- Improved readability for long text blocks
- Magazine and newspaper-style designs
- Better use of wide screens
- Professional typographic appearance
- Automatic text flow balancing
- Responsive layout capabilities

1.2 When to Use column-count

- Long articles or blog posts
- Documentation and guides
- News and magazine content
- Essays and academic papers
- Product descriptions
- Terms and conditions
- Testimonials and reviews

2 column-count Property Basics

2.1 Syntax and Values

```
column-count: <number> | auto;
```

Parameters:

- **<number>**: Exact number of columns (1, 2, 3, etc.)
- **auto**: Let column-width determine columns (default)

2.2 Basic Examples

```
/* Two column layout */
article {
  column-count: 2;
}

/* Three columns */
.content {
  column-count: 3;
}

/* Four column newspaper style */
.newspaper {
  column-count: 4;
}

/* Auto (default) */
div {
  column-count: auto;
}

/* Reset to single column */
.single {
  column-count: 1;
}
```

3 Understanding column-count

3.1 How It Works

- Divides content into specified number of columns
- Text flows top-to-bottom, left-to-right (or RTL)
- Content automatically fills columns evenly
- Columns fill to equal height
- Responsive to container width

3.2 Column Distribution

```
/* Content automatically distributed */
.article {
  column-count: 3;
}

/* HTML: 6 paragraphs */
<div class="article">
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
  <p>Paragraph 3</p>
  <p>Paragraph 4</p>
  <p>Paragraph 5</p>
  <p>Paragraph 6</p>
</div>

/* Result: 2 paragraphs per column */
```

4 column-count with Container Width

Container width affects column appearance significantly.

4.1 Responsive Behavior

```
/* Desktop: 3 columns */
@media (min-width: 1200px) {
  article {
    column-count: 3;
  }
}

/* Tablet: 2 columns */
@media (max-width: 1199px) and (min-width: 768px) {
  article {
    column-count: 2;
  }
}

/* Mobile: 1 column */
@media (max-width: 767px) {
  article {
    column-count: 1;
  }
}
```

4.2 Width Constraints

```
/* Content area width affects column width */
.narrow-content {
```

```
  max-width: 600px;
  column-count: 2;
  /* Each column = ~300px */
}

.wide-content {
  max-width: 1200px;
  column-count: 2;
  /* Each column = ~600px */
}

/* Very wide display */
.ultra-wide {
  max-width: 100%;
  column-count: 4;
  /* Scales with viewport */
}
```

5 Real-World Use Cases

5.1 Use Case 1: Magazine Article

```
/* Magazine-style layout */
article {
  column-count: 2;
  column-gap: 30px;
  column-rule: 1px solid #ddd;
  padding: 20px;
}

article h1 {
  column-span: all;  /* Span all columns */
  font-size: 2.5rem;
  margin-bottom: 20px;
}

article p {
  text-align: justify;
  line-height: 1.6;
}
```

5.2 Use Case 2: Blog Post

```
/* Blog with multi-column text */
.post-content {
  column-count: 2;
  column-gap: 40px;
  font-size: 16px;
  line-height: 1.8;
  max-width: 900px;
}
```

```
/* Single column on mobile */
@media (max-width: 768px) {
  .post-content {
    column-count: 1;
  }
}

/* Byline spans columns */
.post-meta {
  column-span: all;
  font-size: 14px;
  color: #666;
  margin-bottom: 20px;
}
```

5.3 Use Case 3: Documentation

```
/* Technical documentation */
.documentation {
  column-count: 3;
  column-gap: 30px;
  font-family: 'Segoe UI', sans-serif;
}

.documentation h2 {
  column-span: all;  /* Section headers span */
  border-top: 2px solid #333;
  padding-top: 20px;
  margin-top: 30px;
}

.documentation code {
  background-color: #f4f4f4;
  padding: 2px 4px;
  border-radius: 3px;
}
```

5.4 Use Case 4: Testimonials

```
/* Testimonials in columns */
.testimonials {
  column-count: 3;
  column-gap: 20px;
}

.testimonial {
  background-color: #f9f9f9;
  padding: 20px;
  border-left: 4px solid #3498db;
  margin-bottom: 20px;
```

```
  break-inside: avoid;  /* Keep together */
}

.testimonial-author {
  font-weight: 600;
  font-size: 14px;
  color: #333;
  margin-top: 10px;
}
```

5.5 Use Case 5: News Homepage

```
/* News article list */
.news-grid {
  column-count: 2;
  column-gap: 20px;
}

.news-item {
  background-color: white;
  border: 1px solid #ddd;
  padding: 15px;
  margin-bottom: 15px;
  break-inside: avoid;  /* Don't split items */
  border-radius: 4px;
}

.news-item h3 {
  margin-top: 0;
  font-size: 18px;
}

.news-item .date {
  font-size: 12px;
  color: #999;
}
```

6 Related Column Properties

6.1 column-gap

Controls spacing between columns.

```
/* 30px gap between columns */
.article {
  column-count: 2;
  column-gap: 30px;
}

/* Larger gap for readability */
.wide-gap {
  column-count: 3;
  column-gap: 50px;
}

/* Small gap for compact layout */
.compact {
  column-count: 4;
  column-gap: 10px;
}

/* Using em units */
.em-gap {
  column-count: 2;
  column-gap: 2em;
}
```

6.2 column-rule

Adds a visual divider between columns.

```
/* Simple line between columns */
.article {
  column-count: 2;
  column-rule: 1px solid #ddd;
}

/* Thicker rule */
.prominent-rule {
  column-count: 3;
  column-rule: 2px solid #333;
}

/* Dashed rule */
.dashed-rule {
  column-count: 2;
  column-rule: 1px dashed #999;
}

/* Colored rule */
.colored-rule {
```

```
  column-count: 3;
  column-rule: 1px solid #3498db;
}

/* Full column rule property */
.detailed-rule {
  column-count: 2;
  column-rule-width: 2px;
  column-rule-style: dashed;
  column-rule-color: red;
}
```

6.3 column-width

Specifies minimum column width (alternative to column-count).

```
/* Let column-width determine count */
.auto-columns {
  column-width: 300px;
  /* Creates as many columns as fit */
}

/* Combining column-count and column-width */
.flexible {
  column-count: 3;
  column-width: 250px;
  /* Uses up to 3 columns, minimum 250px */
}

/* Wide columns */
.large-columns {
  column-width: 500px;
}

/* Narrow columns */
.narrow-columns {
  column-width: 200px;
}
```

6.4 column-span

Allows element to span across all columns.

```
/* Heading spans all columns */
article h1 {
  column-span: all;
}

/* Default (doesn't span) */
article p {
  column-span: none;
}
```

```
/* Section break spans */
.section-break {
  column-span: all;
  border-bottom: 2px solid #ddd;
  padding: 20px 0;
}

/* Subheading spans */
article h2 {
  column-span: all;
  margin-top: 30px;
}
```

6.5 break-inside

Prevents column breaks inside elements.

```
/* Keep paragraphs together */
p {
  break-inside: avoid;
}

/* Keep images in one column */
img {
  break-inside: avoid;
}

/* Keep entire block together */
.testimonial {
  break-inside: avoid;
}

/* Box stays in column */
.info-box {
  break-inside: avoid;
  background-color: #f0f0f0;
  padding: 15px;
  border-radius: 4px;
}
```

7 Browser Support

7.1 Column Properties Support

Property	Chrome	Firefox	Safari	Edge
column-count	50+	52+	All	79+
column-gap	All	All	All	All
column-rule	All	All	All	All
column-width	All	All	All	All
column-span	All	All	All	All
break-inside	All	All	All	All

7.2 Vendor Prefixes

```
/* With vendor prefixes for older browsers */
.article {
  -webkit-column-count: 2;
  -moz-column-count: 2;
  column-count: 2;

  -webkit-column-gap: 30px;
  -moz-column-gap: 30px;
  column-gap: 30px;
}
```

8 Common Issues and Solutions

8.1 Issue 1: Content Not Flowing

Problem:

```
.article {
  column-count: 3;
  /* Content appears single column */
}
```

Solution:

```
.article {
  column-count: 3;
  width: 100%;  /* Ensure full width */
  max-width: 1200px;  /* Set max width */
}
```

8.2 Issue 2: Elements Breaking in Middle

Problem:

```
.testimonial {
  column-count: 2;
  /* Testimonials split across columns */
}
```

Solution:

```
.testimonial {
  column-count: 2;
}

.testimonial-item {
  break-inside: avoid;  /* Keep together */
}
```

8.3 Issue 3: Uneven Column Heights

Problem:

```
.article {
  column-count: 3;
  /* Columns have different heights */
}
```

Note: This is expected behavior. Columns balance content by default.

Solution:

```
/* Let columns be uneven (natural) */
.article {
  column-count: 3;
  column-fill: auto;  /* Fill columns sequentially */
}

/* Or balance (default) */
.article {
  column-count: 3;
  column-fill: balance;
}
```

9 Responsive Column Layouts

```
/* Base: 1 column */
.article {
  column-count: 1;
}

/* Mobile landscape: 2 columns */
@media (min-width: 600px) {
  .article {
    column-count: 2;
  }
}

/* Tablet: 2 columns */
@media (min-width: 768px) {
  .article {
    column-count: 2;
  }
}

/* Small desktop: 3 columns */
@media (min-width: 1024px) {
  .article {
    column-count: 3;
  }
}

/* Large desktop: 4 columns */
```

```
@media (min-width: 1400px) {
  .article {
    column-count: 4;
  }
}
```

10 Advanced Techniques

10.1 Mixing column-count and column-width

```
/* Use both for flexibility */
.flexible-columns {
  column-count: 3;  /* Maximum 3 columns */
  column-width: 250px;  /* Minimum width */
  gap: 30px;
}

/* If container too narrow, fewer columns appear */
@media (max-width: 900px) {
  .flexible-columns {
    column-count: 2;
  }
}
```

10.2 Column Layout with Images

```
/* Images stay in column */
.gallery {
  column-count: 3;
  column-gap: 20px;
}

.gallery img {
  width: 100%;
  margin-bottom: 10px;
  break-inside: avoid;
  display: block;
}

/* Caption with image */
.gallery-item {
  break-inside: avoid;
}

.gallery-item img {
  width: 100%;
}

.gallery-item figcaption {
  font-size: 12px;
  color: #666;
```

```
  margin-top: 5px;
}
```

10.3 Creating Print-Style Layouts

```
/* Magazine print style */
.magazine {
  column-count: 2;
  column-gap: 40px;
  column-rule: 1px solid #ddd;
}

.magazine h1 {
  column-span: all;
  border-bottom: 3px solid #333;
  padding-bottom: 20px;
  font-size: 2.5rem;
}

.magazine-byline {
  column-span: all;
  font-style: italic;
  color: #666;
  margin-bottom: 20px;
}

.magazine p {
  text-align: justify;
  line-height: 1.8;
}

.magazine figure {
  break-inside: avoid;
  margin: 20px 0;
}
```

11 Performance Considerations

11.1 Performance Impact

- Minimal performance cost
- Browser handles rendering efficiently
- No JavaScript needed
- Works well with large text blocks
- Mobile performance is good

11.2 Optimization Tips

1. Avoid excessive columns on small screens
2. Use break-inside to prevent layout shifts
3. Set max-width for proper column sizing
4. Test on various screen sizes
5. Consider readability (50-75 chars per line)

12 Accessibility Considerations

12.1 Best Practices

- Maintain readable line length (250-400px per column)
- Use appropriate font sizes
- Ensure sufficient line-height
- Test with screen readers
- Provide linear fallback on single column
- Use semantic HTML

12.2 Accessible Examples

```
/* Readable column dimensions */
.article {
  column-count: 2;
  column-gap: 30px;
  font-size: 16px;
  line-height: 1.6;
  max-width: 900px;  /* ~225-300px per column */
}

/* Proper heading hierarchy */
article h1 {
  column-span: all;
}

article h2 {
  column-span: all;
}

article p {
  margin-bottom: 1em;
}
```

13 Summary: Key Takeaways

13.1 Main Properties

Property	Purpose	Values
column-count	Number of columns	Number or auto
column-gap	Space between	Measurement
column-rule	Divider line	Line style
column-width	Min width	Measurement
column-span	Span columns	all or none
break-inside	Break control	avoid or auto

13.2 Best Practices

1. Use column-count for fixed column numbers
2. Pair with column-gap for readability
3. Use column-span for headers
4. Use break-inside to prevent splitting
5. Test responsive behavior
6. Maintain readable line length
7. Consider mobile layouts
8. Use semantic HTML
9. Provide vendor prefixes for older browsers
10. Test accessibility thoroughly

13.3 Quick Reference

```
/* Magazine layout */
.article {
  column-count: 2;
  column-gap: 30px;
  column-rule: 1px solid #ddd;
}

article h1 {
  column-span: all;
}

article p {
  break-inside: avoid;
}

/* Responsive */
@media (max-width: 768px) {
  .article {
```

```
    column-count: 1;
  }
}

/* With column-width */
.flexible {
  column-count: 3;
  column-width: 250px;
  column-gap: 30px;
}
```

14 Conclusion

The CSS `column-count` property enables elegant, magazine-style multi-column layouts without complex HTML or JavaScript. Combined with related column properties, it provides powerful tools for creating professional, readable text layouts.

Key principles to remember:

1. Use column-count for fixed column numbers
2. Combine with column-gap and column-rule for polish
3. Use column-span for headers that should span
4. Use break-inside to prevent awkward breaks
5. Test responsive behavior at all breakpoints
6. Maintain readability with appropriate sizing
7. Consider accessibility and line length
8. Provide vendor prefixes for compatibility
9. Test across browsers thoroughly
10. Use semantic HTML with column layouts

With mastery of column properties, you'll create professional, readable multi-column layouts that work beautifully across all devices and browsers.

Happy coding!